Aarón Fuentes Casanova

Apache



Índice

1. Introducción....3
1.1. Contexto....3
1.2. ¿Qué es Apache y para qué sirve?....3
1.3. Historia de Apache....3
1.4. Alternativas a Apache....3
1.5. Motivación y objetivos....4
2. Instalación de apache....4
3. Creación de una página web....4
4. Control de acceso en Apache....10
5. Resultados y conclusiones....11
5.1 Resultados....11
5.2 Conclusiones....11
6. Bibliografía....12

1. Introducción

1.1. Contexto

El desarrollo y la publicación de aplicaciones web requieren una infraestructura que permita almacenar, gestionar y ofrecer contenido a los usuarios a través de Internet o de una red local. Para ello, los servidores web desempeñan un papel fundamental, ya que reciben las peticiones realizadas por los navegadores y proporcionan los recursos solicitados.

En el ámbito del desarrollo de aplicaciones web, conocer el funcionamiento de los servidores web resulta importante para comprender cómo se publican y se ponen en funcionamiento los sitios web. En esta práctica se estudiará Apache HTTP Server, una de las soluciones más conocidas para este propósito.

1.2. ¿Qué es Apache y para qué sirve?

Apache HTTP Server, conocido habitualmente como Apache, es un software de servidor web de código abierto desarrollado y mantenido por la Apache Software Foundation, junto con su comunidad de colaboradores.

Su función principal es recibir las peticiones de los clientes, normalmente navegadores web, y responder proporcionando los recursos solicitados. Para la comunicación se utilizan principalmente los protocolos HTTP y HTTPS, este último con cifrado para proteger los datos transmitidos.

Entre sus principales usos se encuentran el alojamiento de páginas web, la publicación de documentos e imágenes, la configuración de sitios web y la ejecución de aplicaciones que necesitan un servidor web. Además, dispone de un sistema modular que permite ampliar sus funcionalidades y adaptarlo a diferentes necesidades.

Apache puede utilizarse en distintos sistemas operativos, como GNU/Linux y Windows, y permite configurar aspectos como los puertos de escucha, los directorios desde los que se sirve el contenido, los permisos de acceso y los registros de actividad.

1.3. Historia de Apache

El proyecto Apache surgió en 1995 a partir de los trabajos realizados sobre el servidor web HTTP desarrollado originalmente en el National Center for Supercomputing Applications (NCSA). Un grupo de desarrolladores colaboró para corregir errores, mejorar el software y mantener una versión actualizada, dando lugar al proyecto Apache.

Con el tiempo, Apache evolucionó hasta convertirse en uno de los servidores web más utilizados, gracias a características como su licencia de código abierto, su flexibilidad, su capacidad de configuración y el soporte de una amplia comunidad.

En 1999 se fundó la Apache Software Foundation, una organización dedicada a apoyar el desarrollo colaborativo de proyectos de software libre, entre ellos Apache HTTP Server.

1.4. Alternativas a Apache

Aunque Apache es una solución ampliamente utilizada, existen otras tecnologías que permiten alojar y servir contenido web. Entre las principales alternativas se encuentran:

- **Nginx:** es un servidor web conocido por su arquitectura basada en eventos, que permite gestionar un gran número de conexiones simultáneas. También se utiliza habitualmente como proxy inverso y equilibrador de carga.
- **Microsoft Internet Information Services (IIS):** es el servidor web de Microsoft, integrado en el entorno de Windows Server y utilizado para alojar sitios y aplicaciones web.
- **Caddy:** es un servidor web que destaca por su configuración sencilla y por facilitar la gestión automática de certificados TLS para conexiones HTTPS.

La elección entre estas soluciones depende de factores como el sistema operativo, los requisitos del proyecto, la configuración necesaria, el rendimiento esperado y los conocimientos del administrador.

1.5. Motivación y objetivos

La realización de esta práctica tiene como finalidad conocer Apache HTTP Server y comprender su utilidad dentro del desarrollo y despliegue de aplicaciones web.

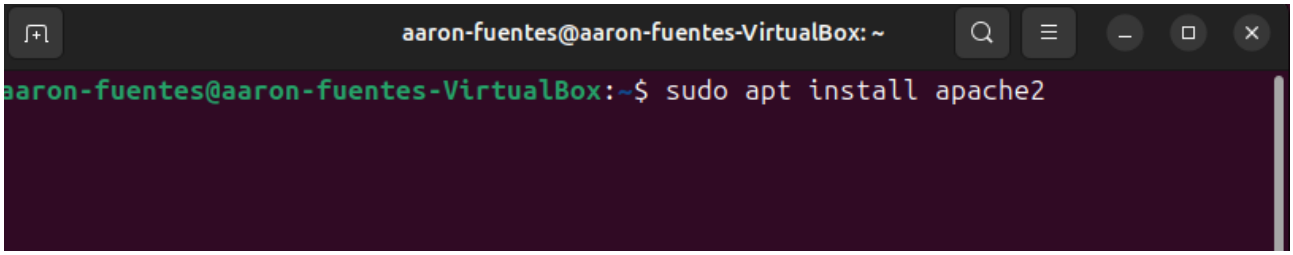
A través de su estudio se pretende adquirir conocimientos sobre el funcionamiento de un servidor web, la comunicación entre clientes y servidores y los principales elementos que intervienen en la publicación de contenido web.

Asimismo, se busca familiarizarse con las posibilidades de configuración de Apache y comprender su importancia en entornos de desarrollo y producción. Estos conocimientos servirán como base para realizar posteriormente tareas prácticas relacionadas con su instalación, configuración y comprobación de funcionamiento.

2. Instalación de apache

En este apartado se va a ver como instalar apache2 en caso de no tenerlo ya.

Para ello hay que poner el siguiente comando: ***sudo apt install apache2***



Una vez que se ejecuta este comando, ya se puede acceder a la carpeta en la que se encuentran los fichero de apache.

3. Creación de una página web

Primero tendremos que acceder a la ruta en la que están los ficheros de apache, que es /etc/apache2.

```
root@aaron-fuentes-VirtualBox:/etc/apache2# ls
apache2.conf    conf-enabled  magic           mods-enabled  sites-available
conf-available  envvars       mods-available  ports.conf    sites-enabled
root@aaron-fuentes-VirtualBox:/etc/apache2#
```

Ahora, cuando accedamos al navegador a http://localhost , podremos acceder a la página por defecto de apache:



Ahora vamos a proceder a crear nuestra primera página web. En este caso va a ser algo muy sencillo, debido a que tan solo estamos probando que apache funcione de manera correcta.

Lo primero que hay que hacer es acceder a la carpeta ***/var/www/html*** , donde van a estar los ficheros html de nuestra página web.

```
root@aaron-fuentes-VirtualBox:/var/www/html# ls
index.html
root@aaron-fuentes-VirtualBox:/var/www/html#
```

Como se puede ver aquí se encuentra el index que usa por defecto apache para mostrar la página al acceder a http://localhost.

Ahora vamos a crear nuestra propia carpeta para nuestra página.

```
root@aaron-fuentes-VirtualBox:/var/www/html# mkdir practica_aaron
root@aaron-fuentes-VirtualBox:/var/www/html# ls
index.html  practica_aaron
root@aaron-fuentes-VirtualBox:/var/www/html#
```

Ya he creado la carpeta en la que van a estar los ficheros de nuestra página web, por lo que vamos a acceder y vamos a crear el primer fichero.

Vamos a crear un host virtual para poder acceder a la página mediante una ruta que le especifiquemos:

```
root@aaron-fuentes-VirtualBox:/etc/apache2# ls
apache2.conf    conf-enabled  magic           mods-enabled  sites-available
conf-available  envvars       mods-available  ports.conf    sites-enabled
root@aaron-fuentes-VirtualBox:/etc/apache2# nano apache2.conf
root@aaron-fuentes-VirtualBox:/etc/apache2# nano ports.conf
root@aaron-fuentes-VirtualBox:/etc/apache2# cd sites-available/
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# ls
000-default.conf  default-ssl.conf
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# nano 000-default.con
f
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# sudo cp 000-default.
conf practica_aaron.conf
```

Lo primero que vamos a hacer va a ser copiar el fichero por defecto de apache y crear el nuestro propio para nuestra página.

Una vez que está creado, vamos a acceder a él:

```
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# ls
000-default.conf  default-ssl.conf  practica_aaron.conf
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# nano practica_aaron.
conf
```

Ahora que estamos dentro vamos a modificar las siguientes directivas:

```
  GNU nano 7.2                    practica_aaron.conf
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port t>
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.
        #ServerName www.example.com

        ServerAdmin aaronfuentes@correoejemplo.ejemplo
        DocumentRoot /var/www/html/practica_aaron
        ServerName aaron.practica.ejemplo.com

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn

                              [ 32 líneas leídas ]
^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea
```

- ServerAdmin: Para poner el correo del administrador en caso de que haga falta que se le envíe alguno.
- DocumentRoot: Para poner la ruta en la que van a estar los ficheros que definen la página web.
- ServerName: La url con la que vamos a poder acceder a la página web.

Una vez hecho esto, vamos a activar el sitio que acabamos de crear. Para ello, hay que poner el comando a2ensite nombreArchivo.conf:

```
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# ls
000-default.conf  default-ssl.conf  practica_aaron.conf
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# sudo a2ensite practi
ca_aaron.conf
Enabling site practica_aaron.
To activate the new configuration, you need to run:
  systemctl reload apache2
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available#
```

Como se puede ver ya he ejecutado el comando, y ahora me pide que reinicie el servicio para que la configuración se efectúe.

```
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# systemctl reload apa
che2
root@aaron-fuentes-VirtualBox:/etc/apache2/sites-available# systemctl status apa
che2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: >
     Active: active (running) since Tue 2026-11-03 19:15:30 CET; 26min ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 7157 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=>
   Main PID: 5182 (apache2)
      Tasks: 55 (limit: 3453)
     Memory: 5.2M (peak: 8.1M)
        CPU: 280ms
     CGroup: /system.slice/apache2.service
             ├─5182 /usr/sbin/apache2 -k start
             ├─7162 /usr/sbin/apache2 -k start
             └─7163 /usr/sbin/apache2 -k start

nov 03 19:15:30 aaron-fuentes-VirtualBox systemd[1]: Starting apache2.service -
nov 03 19:15:30 aaron-fuentes-VirtualBox apachectl[5181]: AH00558: apache2: Cou
```

Como se puede ver en esta captura, ya lo he reiniciado y el servidor está activado.

Ahora, también vamos a tener que añadir el nombre en el archivo hosts dentro de la carpeta /etc. Esto es debido a que como no vamos a acceder a ningún dominio real, tenemos que hacer que

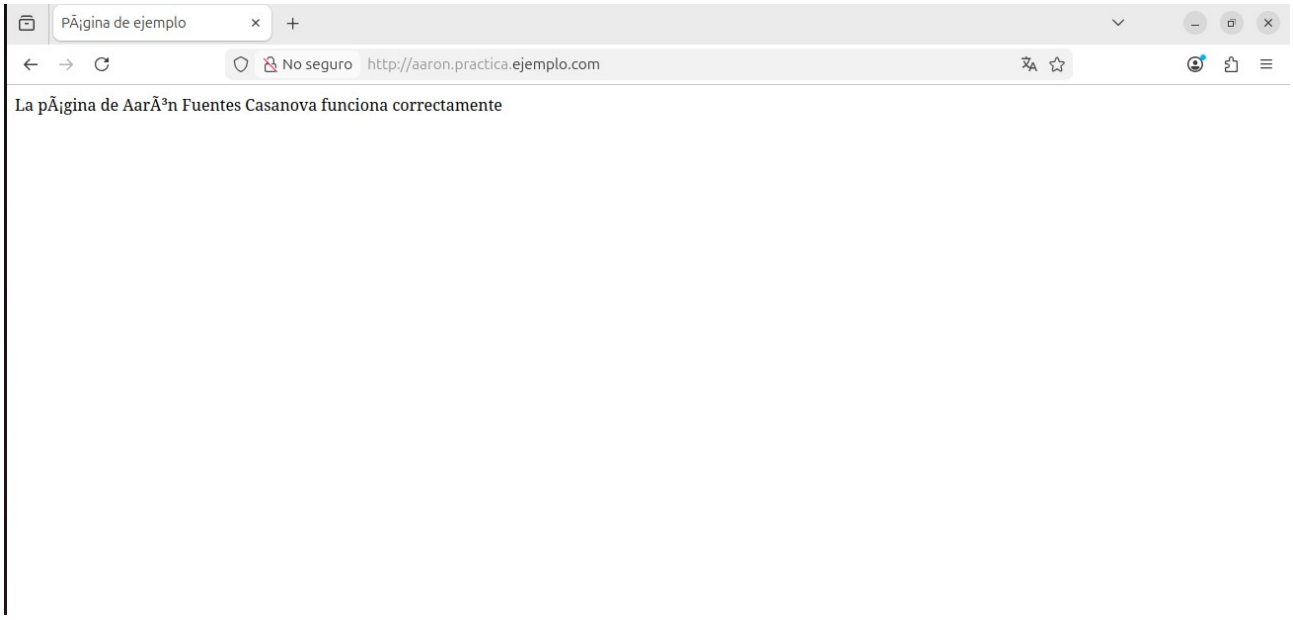
nuestro equipo lo sepa. Para ello vamos a añadir la línea en la que ponemos 127.0.0.1 (localhost) y la ruta de nuestra página:

```
  GNU nano 7.2                        /etc/hosts
127.0.0.1 localhost
127.0.1.1 aaron-fuentes-VirtualBox
127.0.0.1 aaron.practica.ejemplo.com

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
# Added by Docker Desktop
# To allow the same kube context to work on the host and the container:
127.0.0.1       kubernetes.docker.internal
# End of section

                        [ 14 líneas leídas ]
^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea
```

De esta forma una vez que accedemos a la página web (mediante http), podemos ver nuestro archivo index.html



(No se ven las tildes de manera correcta debido a que no está configurado el charset correcto).

Ahora voy a modificar el fichero index.html para que no esté tan vacío el fichero:

```
  GNU nano 7.2                                    index.html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Página de ejemplo</title>
</head>
<body>
    <h1>Página web de Aarón Fuentes Casanova</h1>
    <p>Esta página se ha creado exclusivamente para la realización de la práctica de apache
       de la asignatura Despliegue de Aplicaciones Web.
    </p>
    <p>
       Después de realizar esta página, ya habría terminado la práctica de Apache, tan solo
       me queda documentar los resultados para la memoria que estoy realizando.
    </p>
    <p>
       Aunque primero, antes dar por finalizada la parte práctica, voy a poner un ejemplo
       sobre el control de acceso con Apache.
    </p>
</body>
</html>

^G Ayuda        ^O Guardar      ^W Buscar       ^K Cortar       ^T Ejecutar     ^C Ubicación    M-U Deshacer    M-A Poner marca
^X Salir        ^R Leer fich.   ^\ Reemplazar   ^U Pegar        ^J Justificar   ^/ Ir a línea   M-E Rehacer     M-6 Copiar
```

Y ahora vamos a acceder a la página para ver los cambios que he hecho:



4. Control de acceso en Apache

En este apartado, vamos a ver un ejemplo de como modificar el control de acceso en apache para, por ejemplo en este caso, solo poder acceder desde una dirección ip. Lo primero que vamos a hacer va a ser crear la carpeta en la que va a estar el fichero que va a estar restringido:

```
aaron-fuentes@aaron-fuentes-VirtualBox:~$ sudo mkdir -p /var/www/html/practica_aaron/administracion
```

Y una vez que lo hemos creado, vamos a crear un fichero index.html dentro:

```
  GNU nano 7.2                  /var/www/html/practica_aaron/administracion/index.html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Página de administración</title>
</head>
<body>
    <h1>Zona de administración</h1>
    <p>Acceso restringido.</p>
</body>
</html>
```

Una vez creado, vamos a añadir las siguientes líneas al fichero de configuración de nuestra página:

```
<Directory /var/www/html/practica_aaron/administracion>
        Require ip 192.168.1.1
</Directory>
```

Esto significa que se va a necesitar la ip 192.168.1.1 para acceder a la página de administración.

Cuando no tienes permiso de acceso, te mostrará la siguiente página:



Para tener acceso, he cambiado la ip de 192.168.1.1 para que sea 127.0.0.1 para que siempre que acceda desde localhost tenga permiso para acceder a administración.



Y esto es lo que aparece cuando entramos a esa página.

5. Resultados y conclusiones

5.1 Resultados

En esta práctica, hemos sido capaces de realizar una página web con Apache2, realizar configuraciones de acceso para restringir el acceso de una determinada ruta para que solo puedan acceder las solicitudes que provengan de la ip 127.0.0.1.

Hemos creado una página web con 1 fichero html y una subruta (/administracion), hemos creado el fichero de configuración de la página y cambiado las directivas para que funcionen de manera correcta.

Hemos sido capaces de crear un VirtualHost para poder acceder a nuestra página a través de una determinada url. Para esto había que acceder al fichero /etc/hosts y añadir la ip de localhost junto con la ruta a la que queríamos acceder.

5.2 Conclusiones

En esta práctica hemos visto de manera práctica y sencilla la manera de crear una página web sin muchos problemas. Yo ya había trabajado con Apache anteriormente así que la mayoría de cosas me sonaban, por lo que no he tenido mucho problema. Sin embargo, no me acordaba de la parte de tener que añadir la ruta a /etc/hosts para que funcione el VirtualHost.

Esta práctica siempre me va a parecer algo interesante de ver porque los servidores de apache siguen existiendo y hay bastantes, por lo que el saber como funciona Apache es algo bastante interesante porque nunca se sabe cuando vas a tener que tener que trabajar con un servidor web Apache.

También es algo interesante conocer como funcionan los VirtualHost para poder acceder a las páginas mediante la ruta que pongas y no mediante localhost.

El control de acceso también es importante para poder restringir el acceso a diferentes subrutas de las páginas web, o que solo se pueda acceder desde diferentes rangos de ip si solo quieres que se utilice en una red…

En conclusión, este trabajo me parece bastante interesante para ver como funciona apache y como funcionan los servidores web. En esta práctica se han visto conceptos bastante interesantes y que sin duda sin importantes si vas a trabajar en el desarrollo de aplicaciones web.

6. Bibliografía

- Apache: https://httpd.apache.org
- Apache Software Foundation. *Access Control*. Documentación oficial de Apache HTTP Server 2.4. https://httpd.apache.org/docs/2.4/howto/access.html
- Apache Software Foundation. *Authentication and Authorization*. Documentación oficial de Apache HTTP Server 2.4. https://httpd.apache.org/docs/2.4/howto/auth.html
- Apache Software Foundation. *Apache HTTP Server Version 2.4 Documentation*. https://httpd.apache.org/docs/2.4/
- Ubuntu. Install And Configure Apache. https://ubuntu.com/tutorials/install-and-configure-apache#5-activating-virtualhost-file